Mapeo Hibernate @OneToMany

Desarrollo Entorno Servidor

@OneToMany

Las relaciones 1:N es posiblemente de las relaciones más comunes.

```
CREATE TABLE PROFESOR
(
   ID_PROFESOR BIGINT NOT NULL AUTO_INCREMENT,
   NOMBRE VARCHAR (40) NOT NULL,
   NICKNAME VARCHAR (40) NOT NULL,
   APELLIDOS VARCHAR (40) NOT NULL,
   PRIMARY KEY (ID_PROFESOR)
);
```

```
CREATE TABLE EMAIL
(
   ID_EMAIL BIGINT NOT NULL AUTO_INCREMENT,
   DIRECCION_EMAIL VARCHAR (40) NOT NULL UNIQUE,
   ID_PROFESOR INT NOT NULL,
   PRIMARY KEY (ID_EMAIL),
   CONSTRAINT FK_EMAIL_PROFESOR FOREIGN KEY (ID_EMAIL) REFERENCES PROFESOR (ID_PROFESOR)
);
```

La tabla **Profesor** tiene una relación 1:N con **Email**. Un **Profesor** puede ser relacionado con varios emails.

La columna **ID_PROFESOR** de la tabla **Email** es una clave foránea que se relaciona con la columna **ID_PROFESOR** de la tabla **Profesor**.

Un **Email** no puede existir sin estar relacionado con un **Profesor**. Es por ello que es el **Profesor** el propietario de la relación.

Profesor: Entidad Propietaria

```
public class Profesor implements Serializable {

private Long idProfesor;
```

Profesor: Entidad Propietaria

1. Establecemos las anotaciones básicas.

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {

@Id
@GeneratedValue(strategy=
GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;
```

Profesor: Entidad Propietaria

1. Establecemos las anotaciones básicas.
2. Como un Profesor se relaciona con varios Emails establecemos una variable email como atributo de tipo **Set**.

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {

@Id
@GeneratedValue(strategy=
GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;




private Set<Email> emails = new HashSet<>();
```

Profesor: Entidad Propietaria

1. Establecemos las anotaciones básicas.
2. Como un Profesor se relaciona con varios Emails establecemos una variable email como atributo de tipo **Set**.
3. Anotamos los emails con la anotación **@OneToMany.**

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {

@Id
@GeneratedValue(strategy=
GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;

@OneToMany



private Set<Email> emails = new HashSet<>();
```

Profesor: Entidad Propietaria

1. Establecemos las anotaciones básicas.
2. Como un Profesor se relaciona con varios Emails establecemos una variable email como atributo de tipo **Set**.
3. Anotamos los emails con la anotación **@OneToMany.**
4. Para que sea bidireccional establecemos **mappedBy=** la entidad propietaria es decir Profesor.

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {

@Id
@GeneratedValue(strategy=
GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;

@OneToMany(
mappedBy = "profesor",

                    )
private Set<Email> emails = new HashSet<>();
```

Profesor: Entidad Propietaria

1. Establecemos las anotaciones básicas.
2. Como un Profesor se relaciona con varios Emails establecemos una variable email como atributo de tipo **Set**.
3. Anotamos los emails con la anotación **@OneToMany.**
4. Para que sea bidireccional establecemos **mappedBy=** la entidad propietaria es decir Profesor.
5. Establecemos el borrado en cascada desde el lado propietario con **CascadeType.ALL**

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {

@Id
@GeneratedValue(strategy=
GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;

@OneToMany(
mappedBy = "profesor",
cascade = CascadeType.ALL,
                    )
private Set<Email> emails = new HashSet<>();
```

Profesor: Entidad Propietaria

1. Establecemos las anotaciones básicas.
2. Como un Profesor se relaciona con varios Emails establecemos una variable email como atributo de tipo **Set**.
3. Anotamos los emails con la anotación **@OneToMany.**
4. Para que sea bidireccional establecemos **mappedBy=** la entidad propietaria es decir Profesor.
5. Establecemos el borrado en cascada desde el lado propietario con **CascadeType.ALL**
6. Establecemos **OrphanRemoval**=**True** para que los objetos hijos que no tengan referencia del padre sean borrados.

```java
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {

@Id
@GeneratedValue(strategy=
GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;

@OneToMany(
mappedBy = "profesor",
cascade = CascadeType.ALL,
orphanRemoval = true)
private Set<Email> emails = new HashSet<>();
```

Email: Entidad Poseída

```
public class Email                                    {

private Long idEmail;
```

Email: Entidad Poseída

1. Añadimos anotaciones básicas.

```
@Entity
@Table(name = "EMAIL")
public class Email implements Serializable {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "ID_EMAIL")
private Long idEmail;
```

Email: Entidad Poseída

1. Añadimos anotaciones básicas.
2. Como cada Email solo se relaciona con un Profesor, añadimos un atributo de clase Profesor.

```
@Entity
@Table(name = "EMAIL")
public class Email implements Serializable {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "ID_EMAIL")
private Long idEmail;



private Profesor profesor;
```

Email: Entidad Poseída

1. Añadimos anotaciones básicas.
2. Como cada Email solo se relaciona con un Profesor, añadimos un atributo de clase Profesor.
3. Añadimos la anotación **@ManyToOne**

```
@Entity
@Table(name = "EMAIL")
public class Email implements Serializable {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "ID_EMAIL")
private Long idEmail;

@ManyToOne

private Profesor profesor;
```

Email: Entidad Poseída

1. Añadimos anotaciones básicas.
2. Como cada Email solo se relaciona con un Profesor, añadimos una atributa de clase Profesor.
3. Añadimos la anotación **@ManyToOne**
4. Establecemos que la columna que se usara para hacer el join será ID_Profesor (de la tabla Email) con la anotación **@JoinColumn**

```
@Entity
@Table(name = "EMAIL")
public class Email implements Serializable {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "ID_EMAIL")
private Long idEmail;

@ManyToOne
@JoinColumn(name = "ID_PROFESOR")
private Profesor profesor;
```

Métodos de borrado

Añadimos métodos que faciliten el borrado en la entidad propietaria es decir Profesor.

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {

     public boolean anadirEmails(Email email) {
          email.setProfesor(this);
     return getEmails().add(email);
     }

     public void eliminarEmails(Email email) {
          getEmails().remove(email);
     }
}
```